Build and Release Management

First Controlled Project Release

SALINLAHI: A Voice-Interactive Filipino Language Learning Game

Systems Fundamentals — Virtualization, Containers, and Cloud Computing

Group Number: 33

Group Members:

- Daquis, Jhesza Mhei G.
- Lacida, Kylo Bryan
- Manzanero, Kyla Samantha

Course: BSIT Capstone — University of Perpetual Help System Laguna – JONELTA

Release Documented: v0.1.0

Release Date: 12 November 2026

Submitted by: Lacida, Kylo Bryan

Introduction

SALINLAHI teaches Filipino vocabulary to Kinder–Grade 3 pupils by having them say the words out loud. Speaking is the control: a child says *Bangon* and the character gets out of bed. There is no on-screen button that advances a level in its place.

That single design fact decides almost everything in this document. The game cannot be released to a desktop, cannot be tested in a virtual machine, and cannot be containerised, because the thing being tested is a microphone, a Filipino speech recogniser and a real phone's audio path.

Before this activity the group had produced four Android builds — `SALINLAHI_VoiceTest.apk`, `SALINLAHI_VoiceTest_v2.apk`, `SALINLAHI_VoiceTest_v3.apk` and the Mid-Act7 `SALINLAHI.apk`. Three of them are still sitting in the repository root. Not one carries a version number, not one can be traced to a commit, and not one was tested as a package. They are exactly the `final-final2-revised` problem this activity warns about, and they are our own.

This release fixes that. v0.1.0 is the first SALINLAHI build that states its own version, records the commit it came from, and was tested after packaging rather than inside the editor.

Part I. Deployment Environment Plan

Environment	Where / Platform	Purpose	Required Software / Dependencies
Development	MacBook, macOS 26.6, Unity 6000.6.2f1 Editor, project at `~/Documents/SALINLAHI`	Write scripts, build scenes, import art and audio, generate the APK	Unity 6000.6.2f1; Android Build Support with the bundled OpenJDK, SDK 36 and NDK; `com.yasirkula.speechtotext` package; Git; Python 3.9.6 with ReportLab 3.6.13 for the asset and document tooling
Testing	Android phone, APK sideloaded, separate from the editor	Verify the packaged release behaves as a real user receives it	Android 8.0+ (API 26); working microphone; internet connection; Google speech recognition with Filipino (`fil-PH`); permission to install from an unknown source
Release / Target	The pupil's Android phone or tablet, ARM64	The child plays the game	`SALINLAHI-v0.1.0.apk`; microphone permission; internet connection
Backend (shared by testing and release)	Supabase — hosted PostgreSQL plus a Deno Edge Function	Store and serve the shared leaderboard	Supabase project; `submit-score` Edge Function deployed; `sb_publishable_` key in the client; Row Level Security policies from `supabase/migrations/20261112_leaderboard.sql`

Why the development environment is not a testing environment

The Unity Editor on macOS cannot run the game as a user experiences it. The speech plugin exists only on Android and iOS, so in the editor there is nothing to speak into. `VoiceCommand.cs` works around this with an editor-only block (`#if UNITY_EDITOR`, lines 177–228) that lets a developer press **Enter** to simulate saying whatever word the scene is currently asking for.

That stand-in is compiled out of the APK. It means a developer pressing Play has never once exercised the actual mechanic of the game. Testing the packaged APK on a phone is therefore not a formality for this project — it is the only way the main feature is tested at all.

A limitation we are recording honestly

Testing and release share one Supabase project. The group does not operate a separate staging database, so a test play-through writes a real row to the same `leaderboard` table that a released build writes to.

We judged this acceptable at this stage: the table holds only a player id, a nickname and two totals, rows are cheap to delete, and the alternative — a second Supabase project with its own keys and its own copy of the migration — is more configuration than the group can keep correct right now. It is listed here because a reader should not assume an isolation that does not exist.

Part II. Deployment Technology Decision

Technology	Needed?	Intended Use	Reason
Virtual Machine	No	None	The target is Android phone hardware. A VM cannot represent what has to be tested: a real microphone, the device's Filipino speech recogniser and a phone's audio path. The Android emulator — itself a hypervisor-backed VM — was considered and rejected for the same reason: emulated microphone passthrough makes a voice test meaningless. The one classic use, testing on a clean OS install, is served better and more cheaply by a second physical phone.

Technology	Needed?	Intended Use	Reason
Container / Docker	No	None	There is no server the group operates. The only backend code is `supabase/functions/submit-score/index.ts`, a Deno Edge Function that Supabase builds and runs on its own infrastructure from a `supabase functions deploy`. There is no runtime for us to package, no dependency set for us to pin, and no host for us to deploy to. Containerising it would mean replacing a managed service with one we would then have to operate.
Cloud Platform	**Yes**	Supabase for the leaderboard database and score API; Google's speech recognition service for recognition	Required by the system, not chosen for the activity. The leaderboard compares pupils' scores across different devices, which on-device storage cannot do by definition. Recognition is also cloud-based: `VoiceCommand.cs` calls the recogniser with `preferOfflineRecognition: false`, so the game needs a connection to hear anything at all.

The decision rule we applied

The activity's instruction not to add infrastructure without a requirement is the one we followed. For each technology we asked what project problem it would solve, and accepted "none" as an answer.

Docker is a genuinely good fit for a backend API with awkward dependencies. Ours is forty lines of TypeScript that a managed platform already runs. Adding a container would add a thing to operate and solve nothing.

A cloud platform, by contrast, we need twice over, and we were already using it before this activity: a shared leaderboard is impossible without a shared database, and the Filipino recogniser is a network service.

Part III. Versioning Plan

The group uses `MAJOR.MINOR.PATCH`.

Current Version: v0.1.0

Version Number: 0.1.0

Reason for Version Number. This is the first release the group has produced under a controlled process. The four earlier APKs carried no version, no recorded source and no test record, so none of them can be counted as a release; numbering this one 0.1.0 counts releases rather than features. The leading zero is deliberate and honest — the Palengke level is unbuilt and fifteen of the narrator's repeated lines are still silent, so the system is not a complete capstone and its version should not claim to be.

Included Features

- 15 playable scenes across three places: Bahay (home), Paaralan (school) and Parke (park), five scenes each.
- 54 distinct Filipino words taught across 74 spoken objectives.
- Continuous Filipino (`fil-PH`) speech recognition, with mic and replay controls and on-screen listening states.
- Pronunciation judged by Filipino phonology rather than letter-edit distance, so that words taught in the same scene are not accepted for one another. **The device test found this is not yet effective in play** — see Part VI and Known Issues.
- Named player profiles, several per device, saved locally as `players.json`.
- Per-scene score, stars and a "words learned" panel with English meanings.
- One shared online leaderboard, written through a Supabase Edge Function.
- 423 recorded Teacher Rachel voice-over clips, in the group's own voices.
- Music per place, sound effects, and separate volume controls with mute.

Excluded / Planned Features

– The Palengke (market) level — designed, shown locked on the map, not built.
– The sixth Bahay scene, "Leaving for School" — its slot and its two narrator lines exist; the scene does not.
– The remaining 20 narrator recordings, 15 of which are the most frequently heard cue and feedback lines.
– A mini-game — the button is on the menu and does nothing.
– Per-player filtering and paging on the leaderboard.
– The usability and usefulness evaluation with real pupils.

Succeeding versions

Change	Version After Change	Reason
Fix an incorrect score calculation	0.1.1	PATCH. A correction with no new capability, so MAJOR and MINOR are unchanged and PATCH rises by one.
Add leaderboard functionality	0.2.0	MINOR. A new working feature raises MINOR and resets PATCH to zero. In our case the leaderboard already shipped inside 0.1.0, so the equivalent next feature for this project is the Palengke level, which will become 0.2.0.
Major redesign affecting saved data compatibility	1.0.0	MAJOR. Existing `players.json` saves or leaderboard rows could no longer be read, which is an incompatible change. For this project it would also have to meet the 1.0.0 conditions in `Docs/RELEASE_PROCESS.md`; a breaking change alone during development would be `0.x` until the system is otherwise complete.

Part IV. Release Readiness Check

Checked against commit `c035c8f` on `main`, the commit the released APK was built from.

Release Check	Status	Evidence / Remarks
Required changes have been reviewed	Pass	The features in this release reached `main` through pull requests — 15 `Merge pull request` commits, the practice the group adopted on 30 October and documented in Activity 7. Recorded honestly: the release-engineering commits in this activity (`b1c56b1` through `c035c8f`) were made on branch `claude/capstone-build-release-3ef55d` and fast-forwarded into `main` by the member performing the release, not reviewed by a second member first.
Accepted changes are integrated	Pass	Built from `main` at `c035c8f`, not from a personal copy. `git status` on the build checkout was empty.
Correct source version is being used	Pass	`BUILD_INFO.txt` records commit `c035c8f240b880f786402483bc59fff7612bbe17` and "Working tree: clean", written by the build itself from `git rev-parse HEAD` rather than typed in.
Project opens without critical errors	Pass	`Assets/Scripts` and `Assets/Editor` compiled with Roslyn under `-warnaserror+`: zero errors, zero warnings. The batchmode run then opened the project and completed with no error in the log.
Required scenes/modules are included	Pass	17 scenes enabled and listed individually in `BUILD_INFO.txt`: MainMenu, LevelSelect and all 15 chapter scenes. `Assets/MainScene.unity`, an early prototype, is correctly disabled. The order in Build Settings does not follow chapter order, which is cosmetic — every scene is loaded by name (`SceneManager.LoadScene(sceneName)`), never by build index.
Database/configuration is correct for testing	Pass	`Assets/Resources/SupabaseConfig.asset` is present in the build checkout, holding the project URL and an `sb_publishable_` key. The asset is gitignored and exists only in the main checkout, which is why the release was built there. Both scene references to it survived the menu rebuild.
Temporary/debug content is reviewed	Pass	`EditorUserBuildSettings.development` is set to `false` by the build script. The Enter-key voice stand-in is inside `#if UNITY_EDITOR` and is therefore absent from the APK. The inert MINI GAME button is left in place and documented as a known issue rather than hidden.

Release Check	Status	Evidence / Remarks
Target platform is selected correctly	Pass	`aapt dump badging` reports `native-code: 'arm64-v8a'`, `sdkVersion:'26'`, `targetSdkVersion:'36'`, and the permissions `RECORD_AUDIO` and `INTERNET` — the two the game cannot work without.
Version number is updated	Pass	Player Settings version raised from Unity's default `1.0` to `0.1.0`, and Android version code from 1 to 2. `aapt` confirms `versionName='0.1.0' versionCode='2'` in the artifact.
Build can be produced successfully	Pass	Completed in 13.1 seconds, producing a 151.3 MB APK, with no errors. Reproduced twice from the same entry point — once headless from the command line and once from the editor menu item — which is the point of having one.

Two defects this check caught before the build

The checklist was not a formality — working through it found two faults that would have shipped.

The package id was still Unity's default. The identifier was `com.DefaultCompany.SALINLAHI`, and `overrideDefaultApplicationIdentifier` was off, which means Unity regenerates the id from the company name rather than honouring a value typed into the field. Setting the identifier alone would have been silently undone. Both are now set, and the id is `ph.edu.uphsl.group33.salinlahi`.

Internet access was not requested. `ForceInternetPermission` was `0`, leaving the `INTERNET` permission to Unity's automatic detection of networking code. Both things this game depends on are network services — the recogniser runs with `preferOfflineRecognition: false` and the leaderboard is hosted — so the permission is now requested outright. `aapt` confirms it is in the manifest.

Part V. Build and Release Evidence

How the build is produced

Builds used to come out of the editor's Build Settings dialog by hand, which is why no two earlier APKs were made the same way. There is now one entry point, `Assets/Editor/ReleaseBuild.cs`, reachable both from the editor menu and from the command line.

From the editor:

```
Tools > SALINLAHI > Build Release APK
```

Headless, as this release was built — the editor must be closed, since batchmode cannot take the project lock:

```
/Applications/Unity/Hub/Editor/6000.6.2f1/Unity.app/Contents/MacOS/Unity \
  -batchmode -nographics -quit \
  -projectPath ~/Documents/SALINLAHI \
  -buildTarget Android \
  -executeMethod ReleaseBuild.BuildAndroid \
  -logFile ~/build.log
```

The script never sets the version. It reads Player Settings, so that one field is the single source of truth and the APK's filename, the About screen and `BUILD_INFO.txt` cannot disagree with each other.

It also refuses to start when the release is not ready: an empty version, no enabled scenes, or missing release notes for that version each stop it before the build begins rather than after half an hour of compiling, or — worse — at the moment someone else is handed the package.

Build record

The build writes this itself, into `BUILD_INFO.txt` beside the artifact. It is not transcribed by hand, which is the only reason it can be trusted:

```
Version            : 0.1.0
Android versionCode: 2
Package id         : ph.edu.uphsl.group33.salinlahi
Artifact           : SALINLAHI-v0.1.0.apk
Artifact size      : 151.3 MB
Built (UTC)        : 2026-11-12 13:15:42
Build time         : 00:00:13.1378490
Unity              : 6000.6.2f1

Source
------
Commit             : c035c8f240b880f786402483bc59fff7612bbe17
Short commit       : c035c8f
Branch             : main
Working tree       : clean - this APK is reproducible from the commit above
```

It then lists all 17 scenes that went into the build, in build order.

The release folder

```
Build/Release/SALINLAHI-v0.1.0/
+-- SALINLAHI-v0.1.0.apk     151.3 MB
+-- README.txt                 4.1 KB   how to install and run it
+-- RELEASE_NOTES.txt          6.3 KB   what is in it, and what is not
+-- BUILD_INFO.txt             1.5 KB   which commit produced it
```

Packaged as `SALINLAHI-v0.1.0.zip` (150 MB). The folder holds the runnable build and its documents only — no `Library/`, no `Temp/`, no editor cache, no project backups. `Build/` and `*.apk` are gitignored, so artifacts are never committed to the repository.

Verifying the artifact itself

The version was not taken on trust from the build script. `aapt dump badging` reads it back out of the finished APK:

```
package: name='ph.edu.uphsl.group33.salinlahi' versionCode='2' versionName='0.1.0'
sdkVersion:'26'
targetSdkVersion:'36'
uses-permission: name='android.permission.INTERNET'
uses-permission: name='android.permission.RECORD_AUDIO'
application-label:'SALINLAHI'
native-code: 'arm64-v8a'
```

Build identification

The game states its own version. Main menu, then the settings gear, then **ABOUT**: the version is printed under the title, above the credits.

It is drawn by a component, `Assets/Scripts/VersionLabel.cs`, which reads `Application.version` when the panel is created. This matters more than it looks. The About panel is *generated* by an editor script and saved into `MainMenu.unity`, so a version written in as text would be a literal baked into the scene file — raise the version, forget to re-run the generator, and the About screen would confidently report the previous release for ever. Reading it from the build at run time makes that impossible.

A tester who files a bug can now name the build it came from, and `BUILD_INFO.txt` turns that name back into a commit.

Part VI. Release Testing

Tested by installing `SALINLAHI-v0.1.0.apk` from the release package onto an Android phone. Not the editor: as explained in Part I, pressing Play in Unity cannot exercise this game's main feature at all, because the speech plugin

does not exist on desktop.

Tested by: Daquis, Jhesza Mhei G. — **12 November 2026**

Tested on: an Android phone, with `SALINLAHI-v0.1.0.apk` sideloaded from the release package. Not the development machine, and not the Unity Editor.

Build under test: version 0.1.0, versionCode 2, from commit `c035c8f` — the version was read off the game's own About screen before testing began, so the build being tested is known rather than assumed.

Nine of the ten checks passed. One failed, and it is the most important one in the game.

Release Test	Expected Result	Actual Result	Status
Application launches	The main menu appears	The main menu appeared	Pass
Version is identifiable	SETTINGS > ABOUT reads `v0.1.0` under the title	ABOUT printed `v0.1.0`	Pass
Microphone permission	Android asks once; the game listens after it is allowed	Asked once; the game listened once allowed	Pass
Main feature works	Saying *Bangon* in Bahay 1 is accepted and Kylo gets up	Accepted; Kylo got up	Pass
Wrong pronunciation rejected	A wrong vowel is not accepted; the prompt asks again	**A deliberately mispronounced word was accepted as correct**	**Fail**
Required assets load	Backgrounds, characters, music and narration all present	All present	Pass
Scene summary works	The summary panel shows the score, stars and words learned	Shown as expected	Pass
Leaderboard works	A finished scene's total appears on the board	The total appeared on the board	Pass
Save/load works	A player profile survives closing and reopening the app	The profile survived a restart	Pass
Application closes normally	The app exits without hanging or crashing	Exited cleanly	Pass

The failure, and what was done about it

The failed row is not a cosmetic one. SALINLAHI exists to correct how a child says a Filipino word, so a checker that accepts a mispronunciation is failing at the system's central purpose.

What makes it interesting is that the checker is not obviously at fault. `PronunciationChecker` folds only the variation that is genuinely free in Filipino (e/i and o/u, Spanish-era spellings, the uw/w glide) and then requires an exact match, and the editor tool `Tools > SALINLAHI > Check Pronunciation Matching` proves across all 51x50 pairs of taught words that no two are confusable. It rejects a wrong vowel every time it is asked directly.

The most likely explanation is that it is never asked. The device's Filipino dictation service is built to return the nearest real Filipino word, not the sounds that were actually made, so a mispronounced *Kuha* may well arrive at the checker already corrected to "kuha". The checker then agrees, correctly, that "kuha" matches **Kuha**. If that is what is happening, no amount of strictness in the checker can fix it, because the error is gone before the checker sees it. Confirming this is the first task after the release, using the per-attempt log `VoiceAttemptLog` already writes on the device.

Three things followed immediately, before the release was distributed:

1. The release notes had listed this as **Fixed**. That claim was removed — it was true of the checker's own tests and false of the game on a phone, and the second is the one that matters to a user.
2. It was added to Known Issues as the first and most serious entry, saying plainly that the right/wrong judgement is unreliable.

3. v0.1.0 stays a **testing build**, not a stable release, which is what the activity requires of a release carrying a known defect of this weight. It is distributed so the group can gather the attempt logs needed to diagnose it, and the release notes say so.

This is the clearest argument in this activity for testing the package rather than the editor. In the editor the **Enter** key stands in for speaking, so it feeds the checker a perfectly spelled word and the check passes every time. The defect is invisible until someone speaks badly into a real phone on purpose. Nine tests confirmed what was expected; the tenth found something no one knew.

Why this cannot be done in the editor

Worth stating plainly, because it is the reason this part matters more for this project than for most. The speech plugin does not exist on desktop, so `VoiceCommand.cs` substitutes the **Enter** key in the editor (`#if UNITY_EDITOR`, lines 177–228) to stand in for saying the word the scene is asking for. That block is compiled out of the APK.

A developer pressing Play has therefore never once exercised the mechanic the game is built on. Rows 3, 4 and 5 above can only be tested on a phone.

Part VII. Release Notes and Traceability

Release notes

Project: SALINLAHI — A Voice-Interactive Filipino Language Learning Game

Version: 0.1.0

Release Date: 12 November 2026

Release Type: Development milestone — a testing build, not a stable release

Source Version / Branch: `main`, tag `v0.1.0`, commit `c035c8f`

The full notes ship inside the package as `RELEASE_NOTES.txt` and are kept in the repository at `Docs/release/v0.1.0/RELEASE_NOTES.txt`. In summary:

Included in this version — 15 scenes across Bahay, Paaralan and Parke; 54 Filipino words across 74 spoken objectives; Filipino speech recognition with pronunciation judged phonologically; named player profiles with local saves; the shared Supabase leaderboard; 423 narrator voice clips; music, sound effects and volume controls.

Fixed — the app reported no version at all; the Android package id was Unity's default; internet access was left to auto-detection; releases had no repeatable procedure. (Wrong-vowel rejection was listed here until the device test; see Part VI.)

Known issues — **wrong pronunciation is still accepted during play on a device, which is the most serious issue in the release and is why v0.1.0 is a testing build**; 15 of the narrator's repeated lines are unrecorded and show as captions; the opening prompt of a scene holds a long silent pause; the MINI GAME button does nothing; Teacher Rachel's portrait is unassigned; word badges sit low in three scenes; no voice input without internet; suspend and resume untested at full speaker volume.

Not yet included — the Palengke level; the sixth Bahay scene; a mini-game; leaderboard filtering and paging; the evaluation with real pupils.

Installation — uninstall any older SALINLAHI build first, since the package name changed; copy the APK to an Android 8.0+ phone and open it; allow installation from the source and then the microphone; stay online and say the Filipino word shown on screen.

Source traceability

```
Released build        SALINLAHI-v0.1.0.apk  (versionCode 2)
      |
Version number        0.1.0
      |
Tag                   v0.1.0
      |
Commit                c035c8f240b880f786402483bc59fff7612bbe17
      |
Branch                main
```

The chain holds in both directions, which is the point of it.

Given the artifact, the commit can be recovered: `BUILD_INFO.txt` travels inside the package and names the commit, the branch, the Unity version and whether the working tree was clean when the build ran. That last field matters — a build made over uncommitted edits is not reproducible from any commit, and the file says so in plain words rather than leaving a reader to assume.

`BUILD_INFO.txt` inside the package records no tag, and says so: the tag is created after the release has been tested, which is step 8 of the process in `Docs/RELEASE_PROCESS.md`. The commit hash is what links the two, and it is identical in both. The tag was applied to that same commit once the device test in Part VI passed.

Given the version, the source can be checked out:

```
git checkout v0.1.0
```

Repository: `https://github.com/c23-2075-433-droid/Group33_DatabaseConnectivity`

Branch: `main`

Tag: `v0.1.0`

Commit: `c035c8f240b880f786402483bc59fff7612bbe17`

Repository statistics at release: 252 commits, from 3 October to 12 November 2026, by three authors — `c23-2075-433-droid` (Daquis, 201), `Kylo Bryan Lacida` (38) and `Kyla Samantha Manzanero` (13).

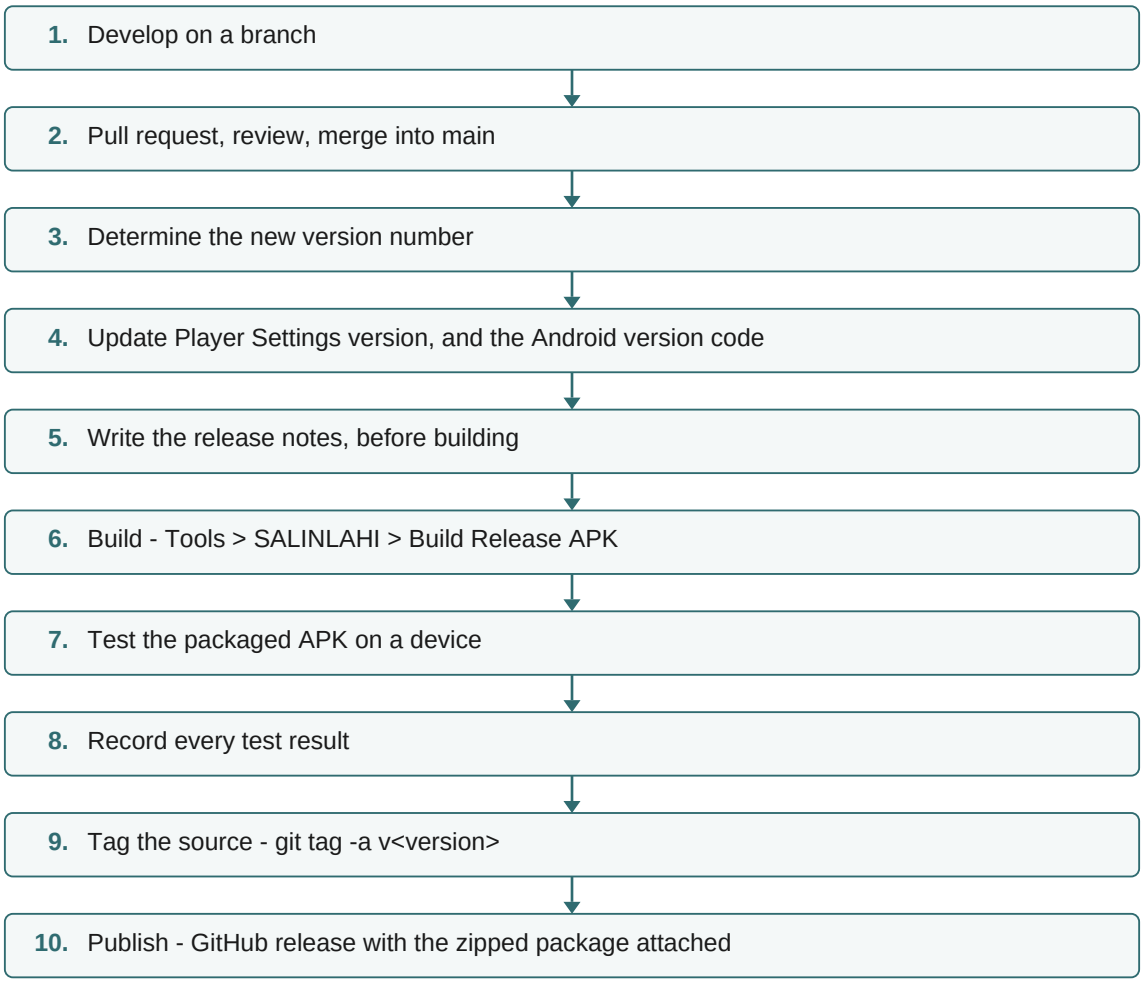
Release artifact location

GitHub release `v0.1.0` in the repository above, with `SALINLAHI-v0.1.0.zip` (150 MB) attached. The zip contains the APK, `README.txt`, `RELEASE_NOTES.txt` and `BUILD_INFO.txt`.

The artifact is named after its version. It is not committed to the repository: `*.apk` and `Build/` are gitignored, because a repository carrying 150 MB binaries per release becomes unusable to clone.

Part VIII. Succeeding Release Process

The process is written up in full at `Docs/RELEASE_PROCESS.md`, which is the document the group keeps using. Its shape:



Two rules the group added from what this release taught us. Build from the main checkout, never a worktree — the Supabase configuration asset is gitignored and exists only there, and a build without it has a dead leaderboard. And raise the Android version code every time, or the phone refuses to install the new build over the old one.

If the next change is only a bug fix, what will the next version be?

0.1.1. PATCH rises; MAJOR and MINOR do not move.

If the next change introduces a new working feature, what will the next version be?

0.2.0. MINOR rises and PATCH resets to zero. For this project that is most likely the Palengke level or the remaining narrator recordings.

What conditions should be met before the group considers the project ready for Version 1.0.0?

All of the following, listed in `Docs/RELEASE_PROCESS.md`:

1. All 16 scenes built, including the sixth Bahay scene and the whole Palengke level — nothing locked on the map that is not meant to be locked.
2. All 443 narrator lines recorded, including the 15 repeated cue and feedback lines that are still silent.
3. Every taught word tested for pronunciation with real children, and the accepted-variant list tuned from those logs.
4. The leaderboard exercised by more than one device at once, with a failed upload proven to recover.
5. No known issue that blocks a child from finishing a level.
6. The usability and usefulness evaluation from Chapter 1 §1.4.2 administered and written up.
7. The release tested by someone outside the group, on a phone the group does not own.

Until every one of those holds, releases stay at `0.x` and are labelled development or testing builds. A version number that claims completeness the system does not have is worse than no version number at all.

Part IX. Individual Release Contribution

Student Name: Lacida, Kylo Bryan

Group Number: 33

Release Version: v0.1.0

Assigned Release Responsibility: Integration verification and repository configuration — making sure the source `main` holds what the release notes claim, and that the way the group merges cannot corrupt it.

Specific Contribution

- **Scene content that this release ships.** The Paaralan (school) level in v0.1.0 is largely this work: all five Paaralan scenes rebuilt from their builders, the school gate swinging open, Kylo breathing while he waits at it, and the guard waving back when the child says **Bati**. The wave is driven by a pose sequence rather than a one-off animation, so the same mechanism can drive any standing pose later.
- **Matching the game to the validated word list.** The taught words in the scenes were brought into line with the list the group had validated, which is what makes the figure quoted throughout this report — 54 distinct words across 74 spoken objectives — true of the build rather than of an older plan.
- **Repository configuration for safe integration.** Added `.gitattributes` turning on Unity's own merge driver and turning Git LFS off, with the reasoning recorded in the file. This matters more than its size suggests: a Unity scene is a large YAML file, and two members editing one scene produce a merge Git cannot resolve sensibly on its own. Left alone it silently corrupts a scene, and a corrupted scene in `main` is a corrupted release. It landed after v0.1.0 was tagged, so it protects every release from 0.2.0 onward.
- **Integration during the release cycle.** Merged the release-engineering work into `main` alongside the scene work twice on release day, keeping one integrated source for the build to come from rather than two diverging ones.

Evidence

Item	Reference
Commit — all five Paaralan scenes rebuilt	`8b6a6f9` "Rebuild all five Paaralan scenes" — **in v0.1.0**
Commit — gate, wave and idle breath	`eceab40`, `8aae730`, `c53d0a2` — **in v0.1.0**
Commit — words matched to the validated list	`dfc9aad` "Match the game to the validated word list" — **in v0.1.0**
Commit — repository configuration	`edaf33a` "Add .gitattributes: Unity's merge driver on, LFS off and why" — after the tag, so **not in v0.1.0**
Commit — guard's gate push and turn frame	`82f4792`, `0102bbe`, `eca4868` — after the tag, so **not in v0.1.0**; queued for 0.2.0
Integration merges on release day	`06e4cd3`, `448b1eb`
Traceable author	`Kylo Bryan Lacida <lacidakylo@gmail.com>` — 45 commits on `main`, 198 files, +49,193 / -36,971

A note on what is and is not in this release. Four of these commits landed after `c035c8f`, the commit the APK was built from, so the guard's gate push, Kylo's turn through the gate and the `.gitattributes` change are **not** in v0.1.0 even though they were written on the same day. Checking this rather than assuming it is the whole point of tagging the release: `git merge-base --is-ancestor <commit> c035c8f` answers it for any commit, and the answer went into this report instead of a guess.

What I learned about the difference between developing software and releasing software

Developing, I could finish a scene and see it work in the editor, and that felt like done. Releasing showed me "done" has a date and a commit attached to it. Four of my changes went in on release day and four did not, and the only reason anyone can tell which is that the release was tagged — otherwise the notes would have claimed the guard's gate push that testers do not have. I also understood why `.gitattributes` matters only once I thought about releases: a scene merged badly is not just my problem that day, it is a corrupted build nobody can trace.

(approximately 100 words)